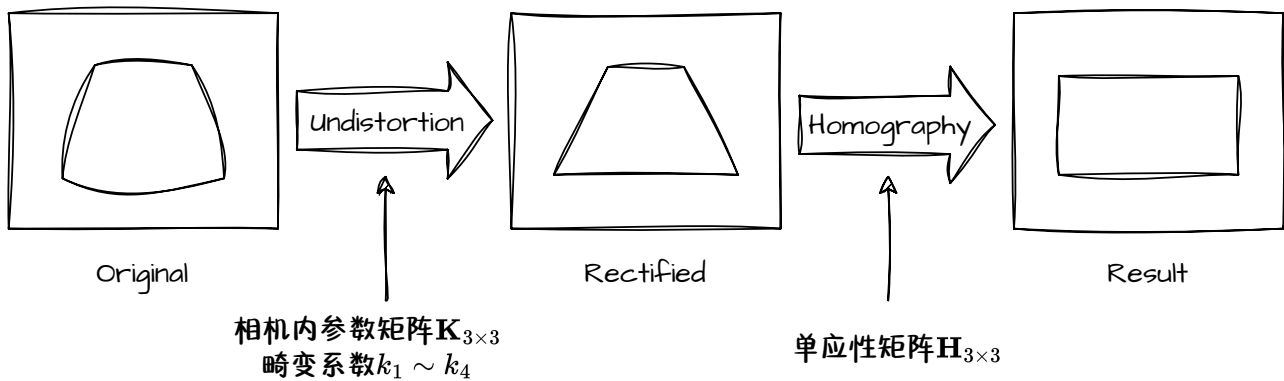


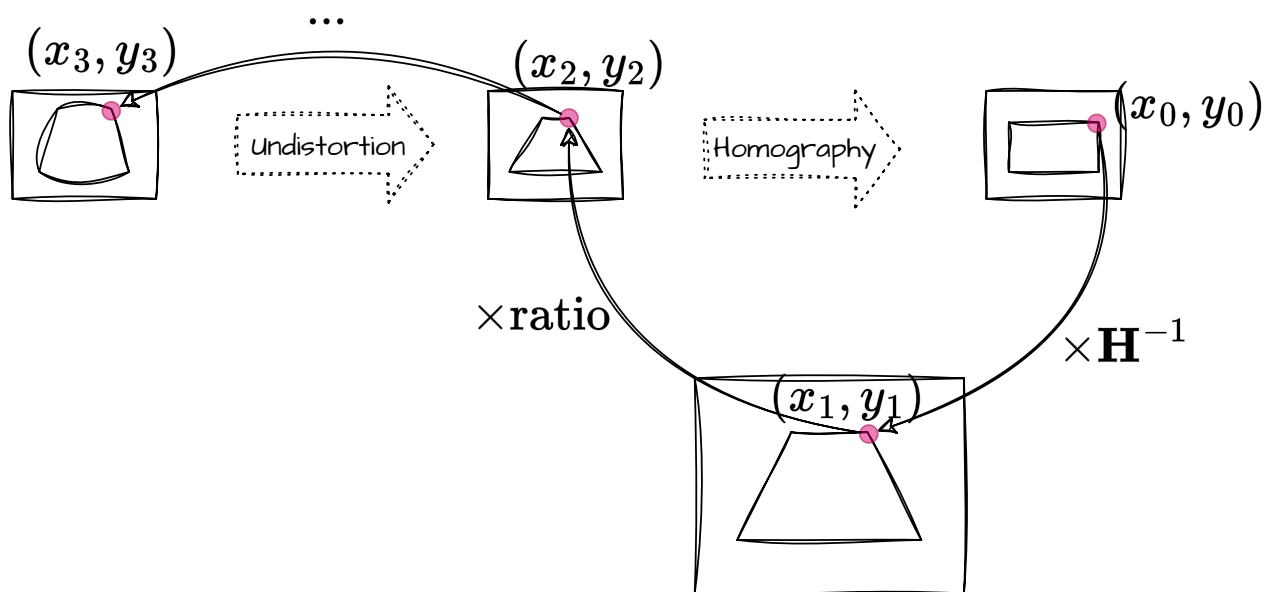
OpenGL实现鱼眼畸变校正和鸟瞰视角变换

以OpenCV实现畸变校正和视角变换的步骤：



在OpenGL中实现以上过程的思路是对Result图像中的每个像素 (x_0, y_0) ，计算它的RGB值来源，即在Original图像中的坐标 (x_3, y_3) 。

考虑输出图像的宽和高均为原始图像的二分之一，即存在一个比例因数ratio的情形。

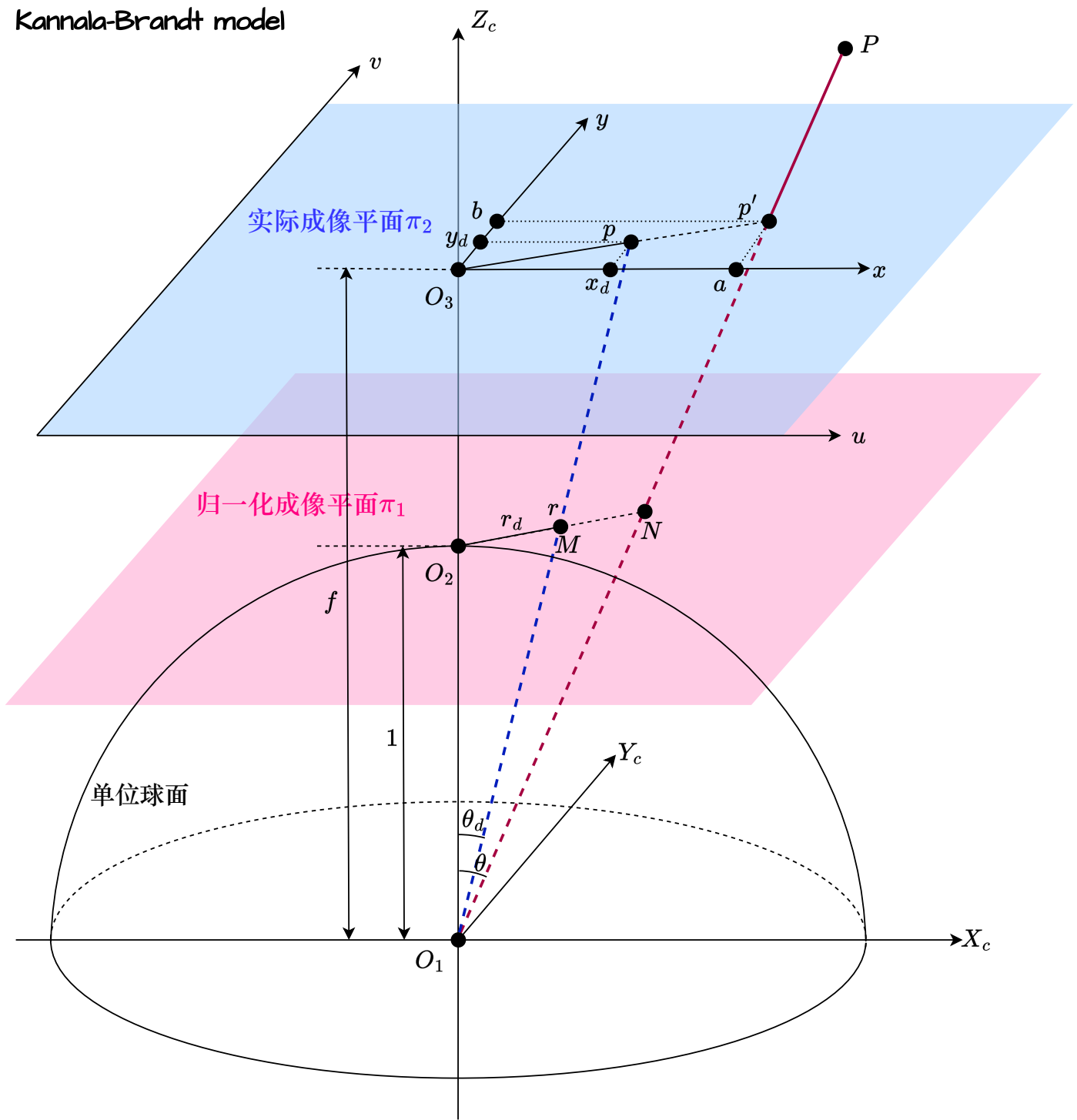


由于这里的单应性矩阵 H 是从“原图尺寸到原图尺寸的对应点映射”计算得到的，所以上图中会存在 (x_1, y_1) 作为中转。如果是基于“缩小后尺寸之间的映射”定义的，则不存在 $(x_0, y_0) \rightarrow (x_1, y_1)$ ，而直接得到 (x_2, y_2) 。

从上面的图示来看，OpenGL的实现过程似乎是OpenCV的逆过程。实则不然。OpenCV中的畸变校正和单应性变换函数的内部实现实际上就是上图的原理，只不过从宏观流程来看，二者似乎是互为逆向的。

鱼眼相机畸变校正算法公式推导

Kannala-Brandt model



光线沿红色虚线射入。如果没有畸变，与两平面分别交于 p', N 两点；但由于鱼眼畸变的缘故，实际落在 p, M 两点。

相机内参数矩阵 $\mathbf{K}$ 将归一化成像平面上的坐标 (x, y) 转换为图像坐标 (u, v)

$$\underbrace{\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{K}} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$$

即

$$\mathbf{K}^{-1} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

因为

$$\mathbf{K}^{-1} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}^{-1} = \begin{pmatrix} \frac{1}{f_x} & 0 & -\frac{c_x}{f_x} \\ 0 & \frac{1}{f_y} & -\frac{c_y}{f_y} \\ 0 & 0 & 1 \end{pmatrix}$$

故有

$$\begin{pmatrix} \frac{1}{f_x} & 0 & -\frac{c_x}{f_x} \\ 0 & \frac{1}{f_y} & -\frac{c_y}{f_y} \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

$$r = \sqrt{x^2 + y^2}$$

$$\theta = \arctan r$$

$$r_d = \theta(1 + k_1\theta^2 + k_2\theta^4 + k_3\theta^6 + k_4\theta^8)$$

在 $\triangle O_1O_2M$ 中

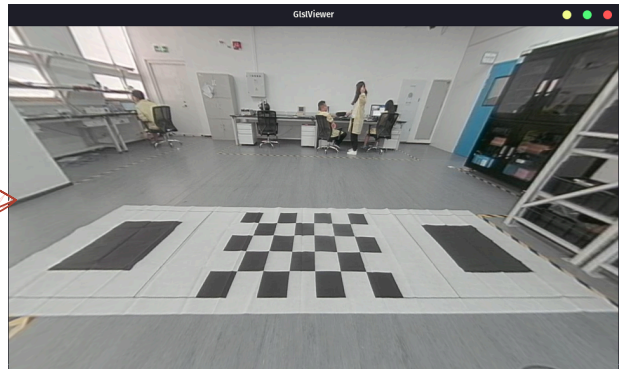
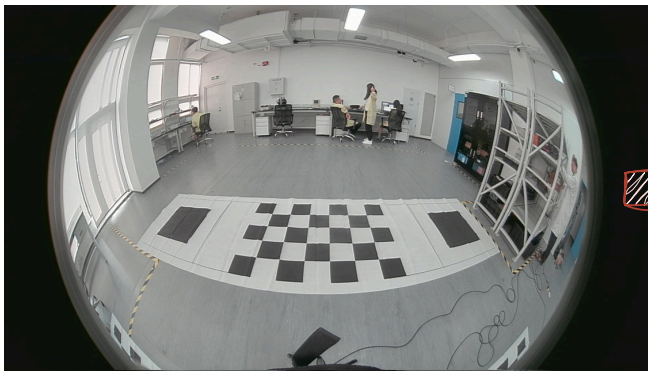
$$r_d = 1 \cdot \tan \theta_d$$

$$\text{scale} = \frac{r_d}{r}$$

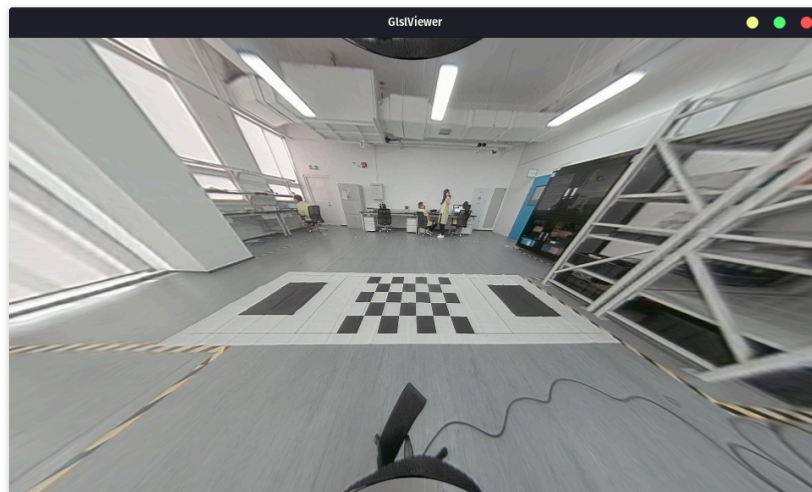
$$\begin{cases} x_d = \text{scale} \times a \\ y_d = \text{scale} \times b \end{cases}$$

$$\begin{cases} u = f_x \cdot x_d + c_x \\ v = f_y \cdot y_d + c_y \end{cases}$$

注：调整畸变校正的scale和shift调整的是 $\mathbf{K}^{-1}$ 中的值，而上式中的 f_x, f_y, c_x, c_y 恒采用相机原始内参数。



畸变校正的视场大小（即直观感受上的“远近”）可以通过scale参数进行调整。以上示例是scale=1的情形。scale=0.5的结果如下：



在shader中要特别注意图像坐标系与纹理坐标系的差异（具体见下一页），并进行适当的转换：

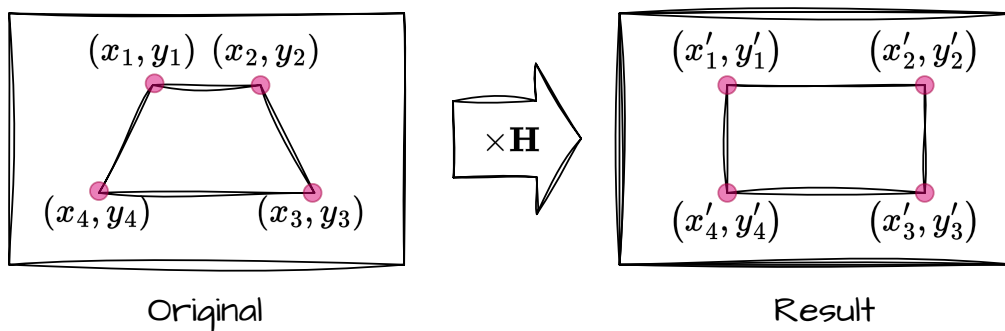
内参矩阵K中的 c_x 和 c_y 表示相机的感光平面中心相对于相机光心的偏移量，单位为像素，且这两个参数是相对于图像坐标系的。然而，畸变校正算法最终希望得到的是纹理坐标系下的坐标，然后再用texture2D()函数取出对应位置的颜色。

为此，需要首先对 c_x 进行以下变换

$$c'_x = \text{resolution.y} - c_x$$

再进行接下来的一系列操作。

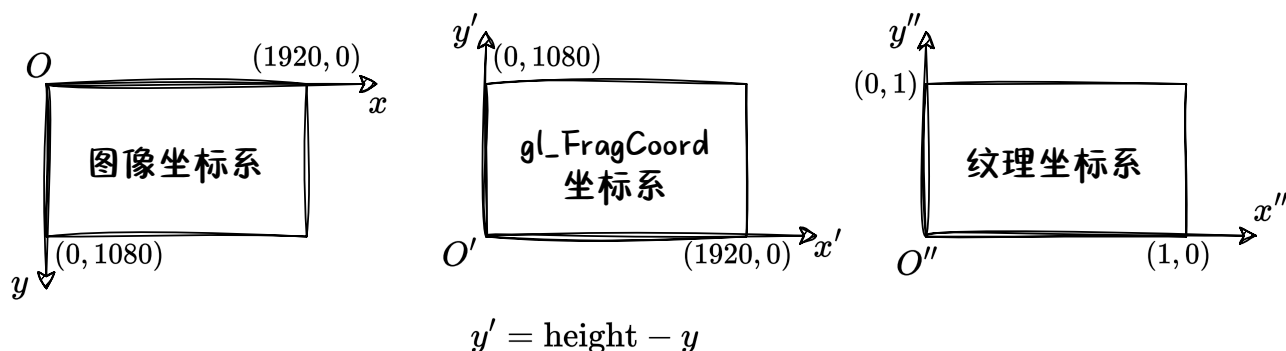
视角变换



$$\mathbf{H}_{3 \times 3} \times \begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} = w_i \begin{pmatrix} x'_i \\ y'_i \\ 1 \end{pmatrix}, i = 1, 2, 3, 4$$

在实际实现中，往往不从原图像出发，而是从待求图像出发，借助单应性矩阵 $\mathbf{H}$ 的逆矩阵 $\mathbf{H}^{-1}$ 来求得每一像素的RGB值。

在OpenGL实现时，要注意gl_FragCoord与图像坐标系的差别，要将特征点的坐标转换到OpenGL坐标系下，再计算 $\mathbf{H}$ 。

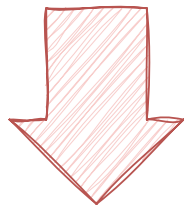
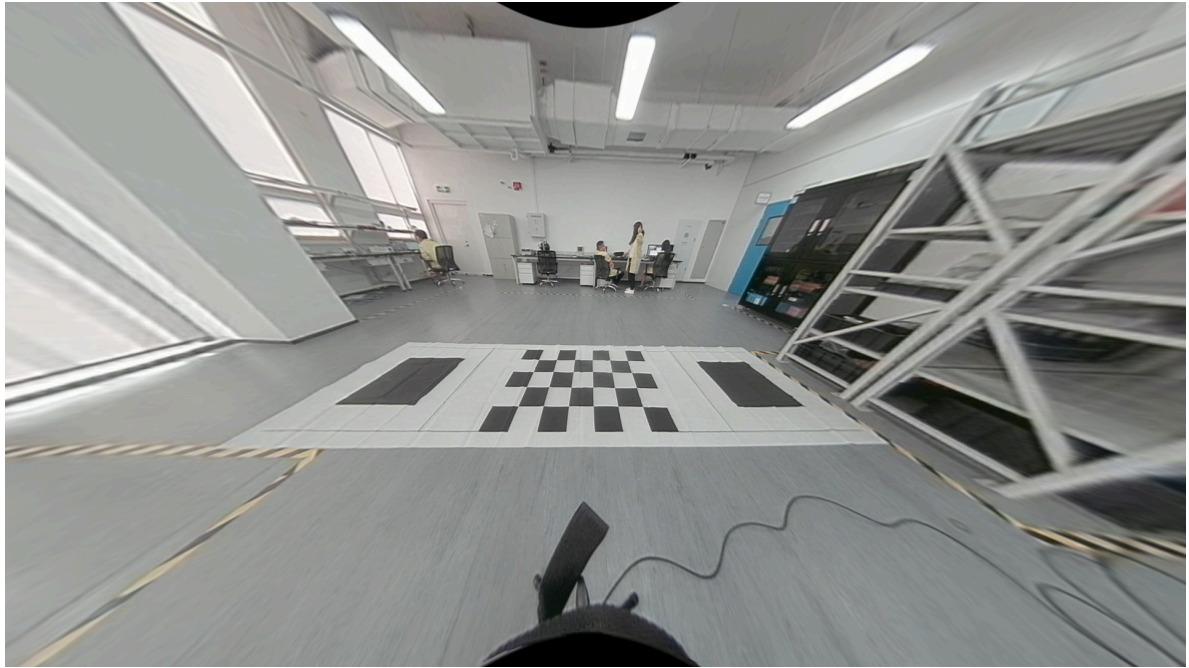


设 $\vec{a} = (x, y)^T$ 和 $\vec{b} = (x', y')^T$ 是一组对应点。如果

$$\vec{a} \xrightarrow{\mathbf{H}_0} \vec{b}, \mathbf{H}_0 = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix}, \mathbf{H}_0^{-1} = \begin{pmatrix} h'_1 & h'_2 & h'_3 \\ h'_4 & h'_5 & h'_6 \\ h'_7 & h'_8 & h'_9 \end{pmatrix}$$

那么

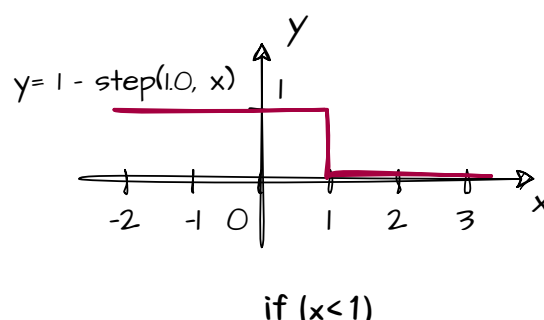
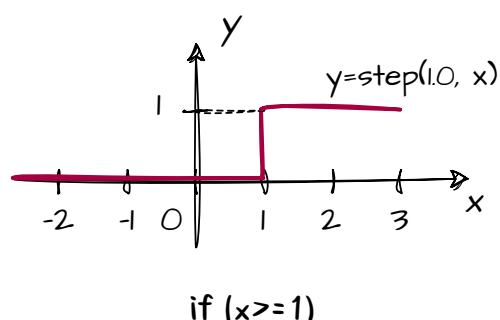
$$\vec{a} \xrightarrow{\mathbf{H}_1} \frac{\vec{b}}{s}, \mathbf{H}_1 = \begin{pmatrix} \frac{h_1}{s} & \frac{h_2}{s} & \frac{h_3}{s} \\ \frac{h_4}{s} & \frac{h_5}{s} & \frac{h_6}{s} \\ h_7 & h_8 & h_9 \end{pmatrix}, \mathbf{H}_1^{-1} = \begin{pmatrix} sh'_1 & sh'_2 & h'_3 \\ sh'_4 & sh'_5 & h'_6 \\ sh'_7 & sh'_8 & h'_9 \end{pmatrix}$$



有效区域裁切

由于着色器程序是在GPU上运行的，所以引入逻辑判断语句可能会显著降低运行效率（具体取决于编译器优化）。更好的做法是利用GLSL中预定义的step函数来迂回实现if语句的功能。

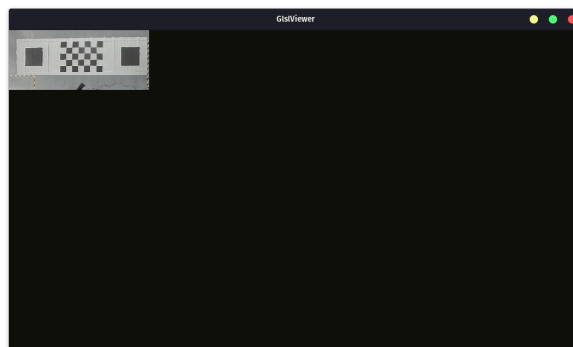
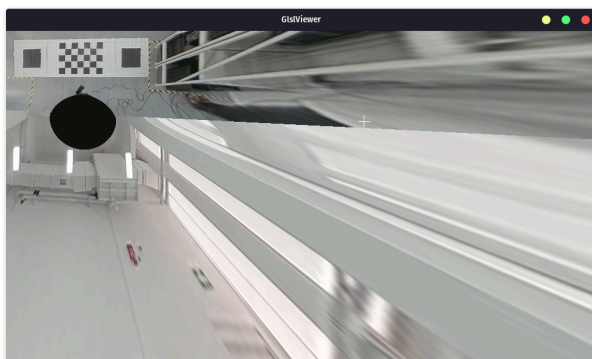
step函数实际上就是单位阶跃函数。它接收2个参数：阶跃位置和待比较值。 $y = \text{step}(1.0, x)$ 的图像如下左图所示，可表示 $\text{if } (x \geq 1)$ ；类似地，可以 $y = 1 - \text{step}(1.0, x)$ 来表示 $\text{if } (x < 1)$ 。



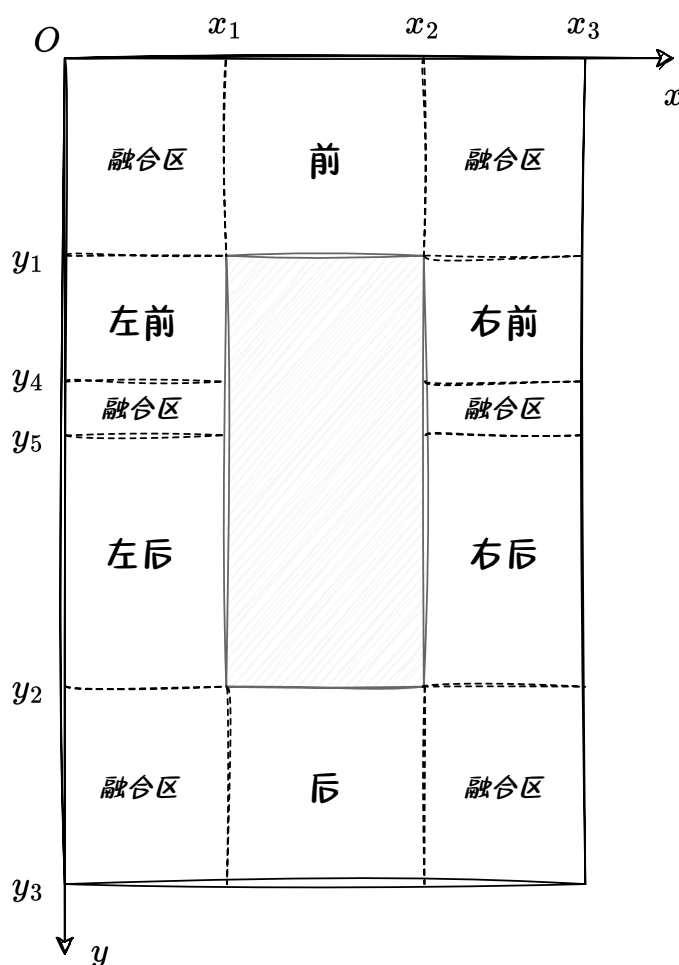
这样，给当前像素的颜色乘以判断语句的值(0或1)，就可以控制显示范围的大小。例如，框定一个矩形的着色器代码如下，其返回值的意义是“当前像素是否在矩形内”。

```
float rectangle_shape(vec2 position, vec2 scale){
    scale = vec2(.5) - .5 * scale;
    vec2 shaper = vec2(step(scale.x, position.x), step(scale.y, position.y));
    shaper *= vec2(step(scale.x, 1. - position.x), step(scale.y, 1. - position.y));
    return shaper.x * shaper.y;
}
```

对经过畸变校正和视角变换的图像，利用以上方法进行裁切的效果如下：



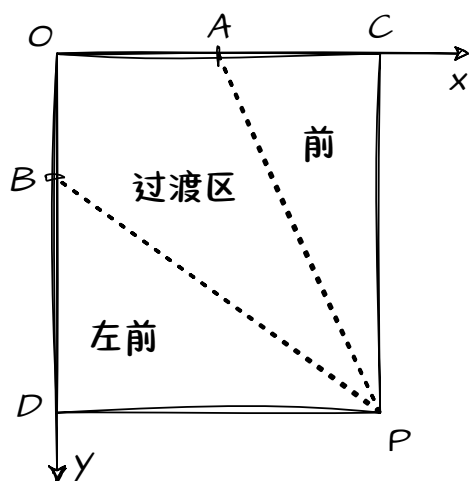
画面布局



这样划分的好处有二：

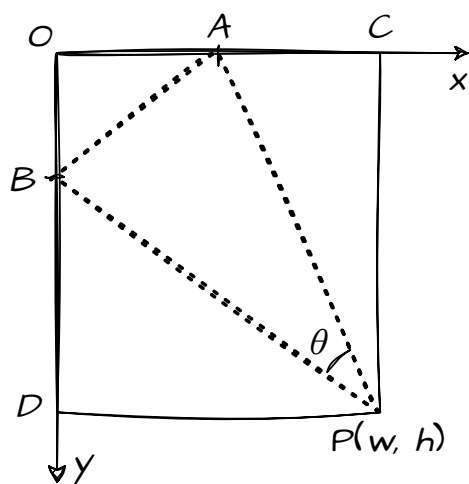
1. 可以分区进行计算和绘制，简化流程。设计两个前端接口，一个处理融合区，一个处理其它区域。最终图像将所有区域的运算结果相加即可，其原理是：因为OpenGL的着色器(shader)程序是对单像素进行运算的，所以借助一系列step()函数可以达到只有当前像素所在区域的对应项不为0的效果。所以，对所有像素都可以一个统一的表达式进行运算。
2. 简化了参数设置步骤。只需设置 $x_1 \sim x_3$ 和 $y_1 \sim y_5$ ，即可确定整个画幅的布局。各个区域的范围 (x_0, y_0, w, h) 以其左上角点坐标 (x_0, y_0) 以及其宽 w 、高 h 确定，通过运算得到其对应向量。

邻接图像融合



1. 左图的矩形区域是前向和左前图像重叠的区域（融合区）。其中， $OABP$ 围成的四边形是过渡区，在这里需要对两幅图像进行融合。这里采用权重蒙板的方法。

$\triangle ACP$ 和 $\triangle BDP$ 分别是权重为1的前向和左前图像。而在过渡区，前向图像的权重需要从 AP 开始的1，以点 P 为圆心旋转，逐渐递减，直到 BP 处减为0。而左前图像的蒙板则与之互补。



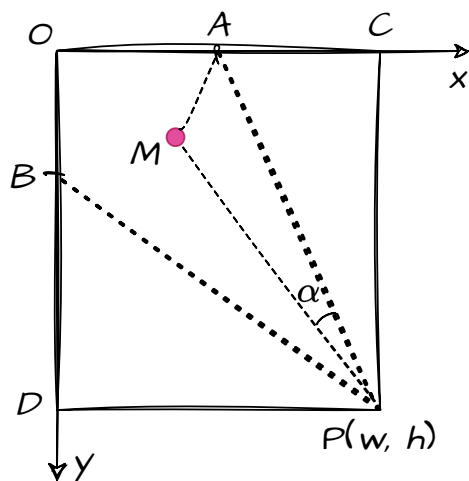
2. 为此，需要先求出 θ 角的大小。在 $\triangle ABP$ 中，由余弦定理可得

$$\cos \theta = \frac{\overline{AP}^2 + \overline{BP}^2 - \overline{AB}^2}{2 \cdot \overline{AP} \cdot \overline{BP}}$$

式中各量都可以方便地求出。

得到 $\angle \theta$ 之后，可求得单位角度对应的权重变化量

$$\Delta w = \frac{1}{\theta}$$



3. 对过渡区的任意一个像素 M ，在 $\triangle MAP$ 中应用余弦定理即可求得 $\angle \alpha$ 的大小。

$$\cos \alpha = \frac{\overline{MP}^2 + \overline{AP}^2 - \overline{AM}^2}{2 \cdot \overline{AP} \cdot \overline{MP}}$$

M 处的权重值即为

$$w_M = 1 - \alpha \cdot \Delta w$$

w_M 乘以255即可以蒙板图像来保存此处的权重值。

这种生成蒙板的方法对画幅四角处的重叠区域均适用。实际上，可以只生成一张蒙板，而通过旋转、翻转的方法得到其它蒙板。

蒙板图像在CPU端只生成一次，通过uniform常量的方式传入OpenGL着色器中。

以MATLAB生成蒙板图像

```
w=200;
h=200;
C=[w,0];
D=[0,h];
P=[w,h];
maskFront=uint8(zeros(h,w,1));
A=[60,0];
B=[0,60];
AB=norm(A-B);
AP=norm(A-P);
BP=norm(B-P);
theta=acosd((AP^2+BP^2-AB^2)/2/AP/BP);
delta_w=1/theta;

for i=1:h
    for j=1:w
        M=[j,i];
        AM=norm(A-M);
        MP=norm(M-P);
        alpha=acosd((MP^2+AP^2-AM^2)/2/MP/AP);
        w_m=1-alpha*delta_w;
        maskFront(i,j)=w_m*255;
    end
end

maskFront=insertShape(maskFront, "filled-polygon", [A;P;C], "Color","white",
    "SmoothEdges",false, "Opacity", 1);
imshow(maskFront);
```



对于画幅左右两侧的融合区来说，过渡方式比较简单，可以方便地用GIMP等绘图工具进行绘制。

注：因为shader中在访问纹理像素值的函数texture2D()是在纹理坐标系下定义的，坐标值是经过归一化的。所以蒙板图像的绝对大小以及长宽比并不重要。



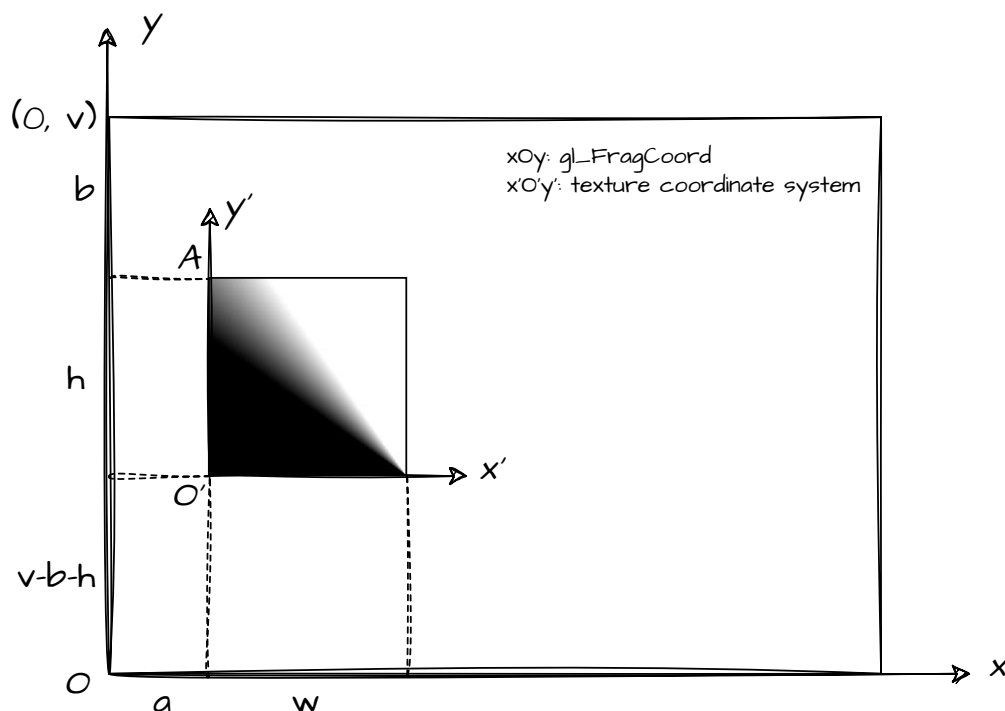
将蒙板图像以uniform常量的方式传入shader后，访问其像素只能在其纹理坐标系（即下图中的 $x'O'y'$ 坐标系）下进行，所以需要将直接拿到的gl_FragCoord坐标转换至其纹理坐标系下。

融合区的位置由其左上角点在gl_FragCoord坐标系下的坐标 $A(a, b)$ 来定义；大小由 (w, h) 定义，如下图所示。

由图可得两个坐标系之间的转换关系

$$\begin{cases} x' = \frac{x-a}{w} \\ y' = \frac{y-(v-b-h)}{h} \end{cases}$$

这样就可以在mask中取得对应的权重值。

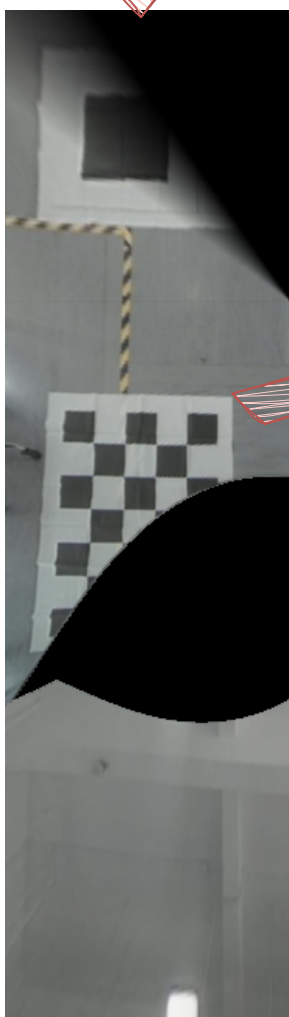
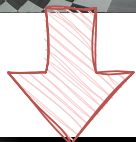


由于这一操作是对画幅中所有像素进行的，而实际上只需要对融合区应用蒙板，所以需要区别对待融合区的内部和外部。和之前“有效区域裁切”部分的方法类似，可以用四个step函数值的乘积来判断当前像素是否在融合区内。

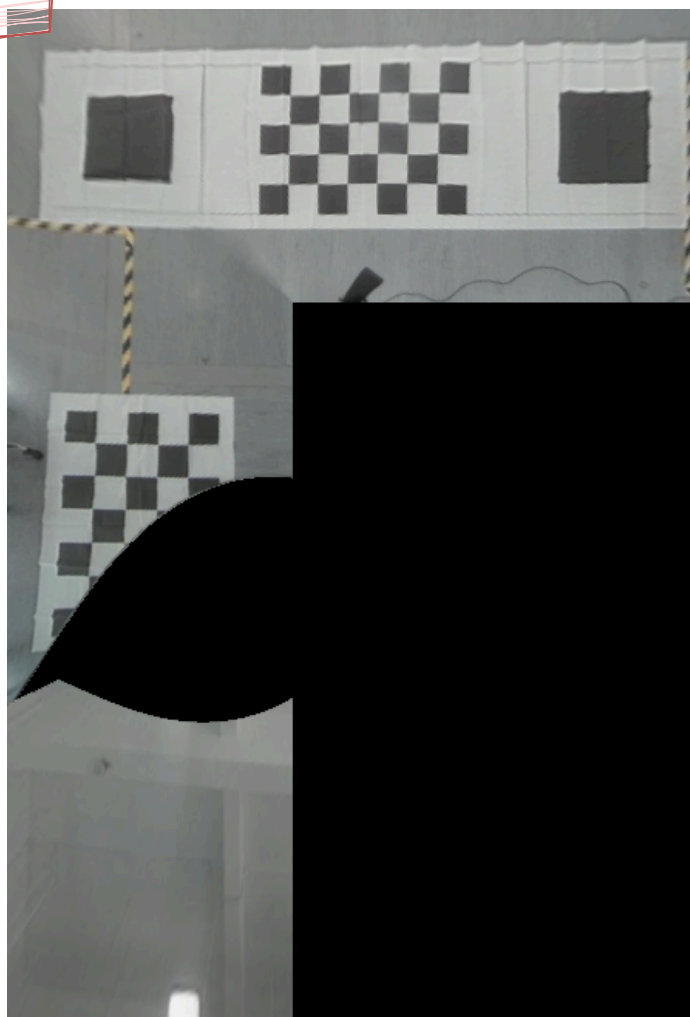
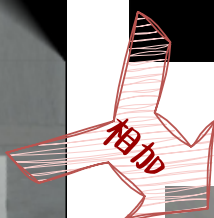
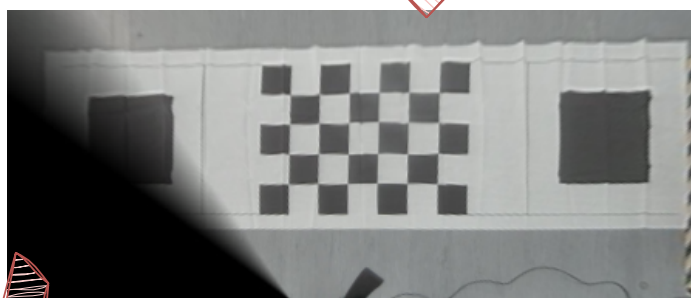
如果以inBlendingRegion表示当前像素是否在融合区内：1表示是，0表示否；maskValue表示从蒙板图中取到的权重值（取值范围为 $[0, 1]$ ），那么来源于不同的两幅图像的权重值 w_1 和 w_2 可以这样定义：

$$\begin{cases} w_1 = \text{inBlendingRegion} \cdot \text{maskValue} \\ w_2 = \text{inBlendingRegion} \cdot (1 - \text{maskValue}) \end{cases}$$

左前

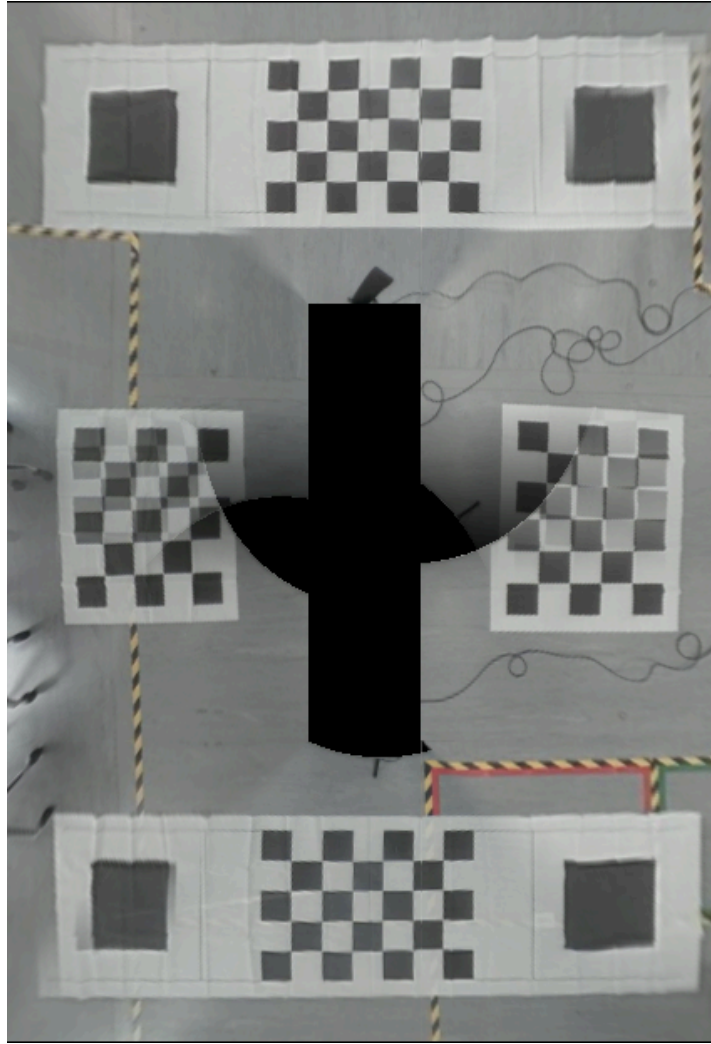


前



最终结果

将所有6幅图像融合，可得到如下图所示的结果：



从上图中可以观察到：

1. 所有融合区均过渡平滑、自然，没有出现跳变的情况。
2. 个别融合区出现的重影来源于标定过程的误差，即在标定布布置、距离测量过程中的不准确。
3. 中央车体两侧出现的黑色部分来源于视野盲区，非算法原因（该组数据并非利用实车采集得到，故相机摆放位置有不合理的情况）。

传参优化

算法用到的所有参数均以一个yaml文件传入。其结构如下：

```
---
# output scale factor & resolution of original images
ratio: 0.9          ratio: 传入shader的图像大小与原始图像大小的比例
resolution: [1920, 1080] resolution: 从相机得到的原始图像大小

intrinsicMatrices:  intrinsicMatrices: 标定得到的鱼眼相机内参数矩阵
  front:      [5.21403992e+02, 0, 9.48270508e+02,
               0, 5.18772705e+02, 5.42287170e+02,
               0, 0, 1]
  leftfront:   [5.24121704e+02, 0, 8.83687500e+02,
               0, 5.21993958e+02, 5.21592651e+02,
               0, 0, 1]
  leftback:    [5.24121704e+02, 0, 8.83687500e+02,
               0, 5.21993958e+02, 5.21592651e+02,
               0, 0, 1]
  rightfront:  [5.24121704e+02, 0, 8.83687500e+02,
               0, 5.21993958e+02, 5.21592651e+02,
               0, 0, 1]
  rightback:   [5.24121704e+02, 0, 8.83687500e+02,
               0, 5.21993958e+02, 5.21592651e+02,
               0, 0, 1]
  back:        [5.21789917e+02, 0, 9.70564880e+02,
               0, 5.19929626e+02, 5.39684631e+02,
               0, 0, 1]

distortionCoefficients: distortionCoefficients: 标定得到的鱼眼相机畸变系数
  front: [-2.4805685241549125e-02, -1.2937205339701639e-
02, 7.5466588447294022e-03, -3.5348453554973126e-03]
  leftfront: [-3.3460359763508023e-02, 1.0100553517874254e-02,
-1.7390230180912274e-02, 5.3465452654110138e-03]
  leftback: [-3.3460359763508023e-02, 1.0100553517874254e-02,
-1.7390230180912274e-02, 5.3465452654110138e-03]
  rightfront: [-3.3460359763508023e-02, 1.0100553517874254e-02,
-1.7390230180912274e-02, 5.3465452654110138e-03]
  rightback: [-3.3460359763508023e-02, 1.0100553517874254e-02,
-1.7390230180912274e-02, 5.3465452654110138e-03]
  back: [-3.0120114488667582e-02, -2.3861699904026055e-03,
-2.7477734953476277e-03, 1.8256980161330622e-04]
```

```

# [x_1, y_1, ..., x_4, y_4]. With respect to the image
coordinate system
# in the original sized (e.g. 1920*1080) image.
featurePoints: featurePoints: 在原始大小图像上选取的四个特征点坐标, 与下面dstPoints对应
front:          [849, 564, 1033, 568, 769, 698, 1095, 698]
leftfront:      [1026, 572, 1196, 555, 1025, 618, 1285, 608]
leftback:       [826, 671, 527, 814, 810, 627, 622, 700]
rightfront:     [731, 634, 858, 564, 697, 714, 882, 599]
rightback:      [1778, 705, 1149, 623, 1490, 576, 1154, 569]
back:           [1194, 756, 815, 754, 1089, 590, 893, 593]

# The corresponding points in the result image
dstPoints: featurePoints: 上面四个特征点落在最终全景拼接图像上的坐标
front:          [170, 40, 310, 40, 170, 140, 310, 140]
leftfront:      [170, 40, 310, 40, 170, 140, 310, 140]
leftback:       [170, 550, 310, 550, 170, 650, 310, 650]
rightfront:     [170, 40, 310, 40, 170, 140, 310, 140]
rightback:      [170, 550, 310, 550, 170, 650, 310, 650]
back:           [170, 550, 310, 550, 170, 650, 310, 650]

# [scale, shift_x, shift_y] parameters for undistortion.
# shift_x: pos=left, neg=right
# shift_y: pos=up, neg=down
undistortionScaleShift: undistortionScaleShift: 畸变校正的尺度和偏移量参数。可控制
front:          [0.5, 0, 0] 视场大小和观察位置
leftfront:      [0.5, 0, 0]
leftback:       [0.5, 0, 0]
rightfront:     [0.5, 0, 0]
rightback:      [0.5, 0, 0]
back:           [0.5, 0, 0]

# Layout in the result image.
layout: layout: 布局参数
vehicle: layout.vehicle: 车的大小和位置。(x1, y1)左上角, (x2, y2)右下角。
x1: 200
x2: 275
y1: 200
y2: 490
canvas: layout.canvas: 画布 (即最终图像) 大小
x3: 470
y3: 690
flankBlendingAreas: layout.flankBlendingAreas: 左右两侧融合区的高度和位置
y4: 265
y5: 380

```

Fragment shader code

Standalone version (undistortion + homography):

```
#ifndef GL_ES
precision mediump float;
#endif

uniform sampler2D u_tex0;
uniform vec2 u_resolution;

vec2 fisheye_undistortion(vec4 k, mat3 K, float scale, vec2 shift, vec2 coord, float ratio){ // k: distortion coefficients;
K: intrinsic matrix
K[0][0]*=ratio;
K[1][1]*=ratio;
K[2][0]*=ratio;
K[2][1]=u_resolution.y/u_ratio-K[2][1]; // transform c_y of K to the texture coordinate system
K[2][1]*=ratio;
float
fx_inv=1./(K[0][0]*scale),
fy_inv=1./(K[1][1]*scale),
cx_inv=-(K[2][0]+shift.x)/K[0][0],
cy_inv=-(K[2][1]+shift.y)/K[1][1];
mat3 K_inv=mat3(fx_inv,0.,0.,0.,fy_inv,0.,cx_inv,cy_inv,1.); // col major

float
i=coord.y, // row number
j=coord.x, // col number
_x=i*K_inv[1][0]+j*K_inv[0][0]+K_inv[2][0],
_y=i*K_inv[1][1]+j*K_inv[0][1]+K_inv[2][1],
_w=i*K_inv[1][2]+j*K_inv[0][2]+K_inv[2][2];
float
x=_x/_w, y=_y/_w,
r=sqrt(x*x+y*y),
theta=atan(r),
theta2=theta*theta,
theta4=theta2*theta2,
theta6=theta2*theta4,
theta8=theta4*theta4,
r_d=theta*(1.+k[0]*theta2+k[1]*theta4+k[2]*theta6+k[3]*theta8),
r_scale=r_d/r; // if r=0 then r_scale=1. Ignore this rare case for performance reason
float
u=K[0][0]*x*r_scale+K[2][0],
v=K[1][1]*y*r_scale+K[2][1];
// Here in undistortion, size(OutputImg)=size(InputImg), meaning max(u)=u_resolution.x, max(y)=u_resolution.y.
// It's because the input coord here were gotten from homography_transformation(), ranging from vec2(0)~u_resolution*ratio.
// (since H_inv has got modified w/ ratio)
// u&v will be normalized to [0,1], and thus ignore the image size and can be used to retrieve color from the original image.
return vec2(u,v);
}

vec2 homography_transformation(mat3 H_inv, vec2 coord, float ratio){
vec3
dst=vec3(coord,1.),
src=H_inv*dst,
src_normalized=src/src.z;
return src_normalized.xy*ratio; // since H was defined as [W,H]->[W/2,H/2]
}

float cropping(vec4 cropRange, float ratio){ // cropRange.y is under image coordinate system
float
x=cropRange.x*ratio,
y=cropRange.y*ratio,
w=cropRange.z*ratio,
h=cropRange.w*ratio,
withinRange=step(x,gl_FragCoord.x)*(1.-step(x+w,gl_FragCoord.x))
*step(y,gl_FragCoord.y)*step(u_resolution.y-(y+h),gl_FragCoord.y);
return withinRange;
}

// Wrapper: undistortion + homography + cropping
vec2 birdView(vec4 k, mat3 K, float scale, vec2 shift, mat3 H_inv, float ratio, vec2 coord, vec4 cropRange){
vec2 resultCoord=homography_transformation(H_inv,coord,ratio); // coord in undistorted
resultCoord=fisheye_undistortion(k,K,scale,shift,resultCoord, ratio); // coord in original
float homographyWithinRange=step(0., resultCoord.x)*(1.-step(u_resolution.x,resultCoord.x))
*step(0., resultCoord.y)*(1.-step(u_resolution.y,resultCoord.y));
// float croppingWithinRange=1.;
float croppingWithinRange=cropping(cropRange, ratio);
// return resultCoord;
return resultCoord*homographyWithinRange*croppingWithinRange;
}
```

```

void main(){
float ratio=u_resolution.x/1920.; // ratio = window_resolution / original_image_size
/// Calibration parameters
float fx=5.21403992e+02,fy=5.18772705e+02,cx=9.48270508e+02,cy=5.42287170e+02;
vec4 distortionCoefficients=vec4(-2.4805685241549125e-02,-1.2937205339701639e-02,7.5466588447294022e-03,-3.5348453554973126e-03);
mat3 intrinsicMatrix=mat3(fx,0.,0.,0.,fy,0,cx,cy,1.);//column major
// Undistortion parameters
float scale=.5;vec2 shift=vec2(u_resolution.x*0.5,u_resolution.y*0.5);
/// Homography transformation
/*
/>\ H is culculated with points under the gl_FragCoord's coordinate system, which means that for all the points (in both src
& dst imgs) used to calculate M,
their y*=-1, y+=imgHeight. Then the inverse of M is used to find out where the current pixel's color is originated. /\
/>\ Also, the coordinates of the source points used to get H are from the original sized image, without shrinkage. /\ */
mat3 H_inv=mat3(6.47099220213213,.352476689179285,.000922713846019077,
17.6105264345445,12.7053779308605,.0185329623342456,
-15850.6187148092,-11107.1665175717,-15.2327941820516);
H_inv[0]/=ratio;
H_inv[1]/=ratio; //M for [src_points -> dst_points*ratio].

/// Crop range (under image coordinate system)
vec4 cropRange=vec4(0,0,470,200);

/// original (distorted) -> undistorted -> birdview (result))
// vec2 resultCoord=birdView(distortionCoefficients, intrinsicMatrix, scale, shift, H_inv, ratio, gl_FragCoord.xy,
cropRange);
vec2 resultCoord=fisheye_undistortion(distortionCoefficients,intrinsicMatrix,scale,shift,gl_FragCoord.xy,ratio);

vec2 textureCoord=resultCoord/u_resolution;
gl_FragColor=texture2D(u_tex0,textureCoord);
}

/// ===== Visualization using glslViewer =====
// ratio=0.25
// glslViewer undistortion_opencv_0802front.frag front.jpg -l -w 480 -h 270
// ratio=0.5
// glslViewer undistortion_opencv_0802front.frag front.jpg -l -w 960 -h 540
// ratio=1
// glslViewer undistortion_opencv_0802front.frag front.jpg -l -w 1920 -h 1080

```